

Validation Safeguards for Bulk Editing

Last Updated: 2026-10-02

Applies to `Module_Receiving` Edit Mode and Batch Edit, and to `Module_Reprint` batch queueing.
Written for `Module_Shared` because the reusable pieces are the shared lookup control
(`Module_Shared/Views/Controls/Control_Shared_TypedLookupTextBox`) and the shared view-model base
(`ViewModel_Shared_Base`).

Scope

The problem this document solves: a batch edit applies one typo, one fuzzy match, or one restricted part to forty skids at once. Row-at-a-time editing fails safely because the operator sees each row. Batch editing fails catastrophically. The safeguards below exist to make the multiplicative failure mode impossible rather than merely unlikely.

Principles

1. **Validate before the commit, not after.** `SaveAsync` already validates at save time. The batch overlay must surface the same findings while the operator is still deciding, so a blocker is never discovered after the dialog closes.
2. **Cheap checks synchronously, expensive checks asynchronously.** Format, blank, and range checks run inline. Anything that needs the database is debounced and cancellable.
3. **Every finding names its rows.** "Invalid part" is useless with a selection of 3. " `MMC00076` is not an exact part match (1 skid: `V-F0-03`)" is actionable.
4. **Ambiguity is never resolved silently.** A fuzzy match in a single-row edit is a convenience; in a batch it is a liability.
5. **The UI never freezes.** No synchronous database work on the UI thread, ever, for any validator.
6. **Failures are partial, and partial failures are reported per row.** No silent skipping.

Live Validation Pipeline

- Operator types into an overlay field
- > 1. Debounce (300 ms) and cancel the previous request
 - > 2. Synchronous cheap checks (blank / format / sign / range)
 - > 3. Asynchronous checks (lookup, restricted part, ownership, queue conflict)
 - > 4. Reduce findings into a severity bucket
 - > 5. Publish to: inline field status, validation pane, Apply button gate, preview row status

Field-by-field validation contract

Field	Check	Source	Timing	Severity when it fails
Part ID	Exact match exists	<code>Control_Shared_TypedLookupTextBox</code> , <code>LookupType="PartNumber"</code> , <code>ValidationCompleted</code>	Debounced async	Block
Part ID	Fuzzy fallback used	<code>Model_SharedLookupValidationResult.UsedFuzzyFallback</code>	Async	Confirm — never auto-apply
Part ID	Value was reformatted	<code>Model_SharedLookupValidationResult.WasFormatted</code> plus <code>FormattedValue</code>	Async	Info — show the reformatted value, do not block

Field	Check	Source	Timing	Severity when it fails
Part ID	Restricted part / quality hold	<code>IService_ReceivingValidation.IsRestrictedPartAsync</code>	Async, one call for the batch value, not one per row	Confirm with an explicit acknowledgement, mirroring <code>SaveAsync</code> 's <code>ShowQualityHoldConfirmationAsync</code>
Part ID	Same as current value	Local comparison	Sync	Info — "no change for N skids"
Heat / Lot #	Blank	Local	Sync	Warn — normalized to Nothing Entered at save, matching existing behavior
Location	Exact match in warehouse 002	Typed lookup, <code>LookupType="Location"</code> , <code>WarehouseCode="002"</code>	Debounce async	Block
Location	Overflow / locked location	Location lookup result	Async	Block
Weight / Qty	Negative result for any row	Local arithmetic across the preview set	Sync	Block
Weight / Qty	Distribute produces a negative remainder	Local arithmetic	Sync	Block
Weight / Qty	Aggregate vs expected total	Local	Sync	Warn or Confirm, per the reconciliation tolerance rules
Pkgs / Load	Non-integer, negative, or zero	Local	Sync	Block
Ownership	Operator cannot manage one of the rows	<code>EnsureCurrentUserCanManageLoadsAsync</code> equivalent, called once with the full row set before apply	Async, on dialog open and on selection change	Block
Reprint conflict	Row already queued with <code>is_reprint = 1</code>	Reprint history / <code>AlreadyQueued</code>	Async, before the print step	Warn with the queuing timestamp

Ownership and quality-hold checks are deliberately **per-batch, not per-row**. Calling `IsRestrictedPartAsync` forty times for forty rows that are all being set to the same part is a needless forty round trips; call it once with the new value.

Debounce, cancellation, and stale responses

The shared lookup control already exposes `IsValidatingInProgress` . Build on it rather than adding a second busy concept.

```
Field input changes
-> cancel the CancellationTokenSource for this field
-> start a new CTS, await Task.Delay(300, token)
-> run the async check with the token
-> if the token is cancelled, discard the result and return without touching the UI
-> if the field's value changed while the check was in flight, discard the result
```

The second guard matters more than the first. A slow exact-match response for `MMC00065` must not overwrite a fast response for `MMC000767` .

Mixed-value and batch sizing rules

- A field left unticked is never validated for value correctness, only for consistency with the batch (for example, all unticked rows must still be non-negative).
- Validation runs against the **new value**, not per row, for single-value overrides.
- Validation runs per row only for operations that produce row-specific results: `Add` , `Subtract` , `Scale by` , and `Distribute` .

Severity Model

Severity	Meaning	Effect on the Apply button	Presentation
Block	The batch cannot be applied as configured	Disabled	Red inline field message plus a red validation pane entry
Confirm	Applicable, but the operator must acknowledge	Enabled, gated behind an acknowledgement checkbox in the validation pane	Amber inline message; acknowledgement required once per batch, not per row
Warn	Worth noticing, does not change the outcome	Enabled	Amber inline message; dismissible
Info	Neutral information about the batch	Enabled	Grey inline message

Exactly one `Block` finding disables Apply. The pane always lists every finding, not just the first, so the operator fixes the batch in one pass instead of discovering problems serially.

What the Operator Sees, and When

Moment	Surface
Typing in a field	Inline status under that field: spinner while <code>IsValidatingInProgress</code> , then a glyph plus one line of text
Any finding present	Validation pane <code>InfoBar</code> above the preview, summarizing counts by severity
Both counts non-zero	Pane reads "2 issues need attention, 1 must be fixed" rather than showing only the blocker
Apply pressed with blockers	Apply is already disabled; the button tooltip names the blocker count
Apply pressed with confirms pending	Apply stays enabled, but the click is intercepted, the acknowledgement checkbox is focused, and the button does nothing until it is ticked
Apply pressed successfully	Dialog closes; findings are preserved on the view model so the Selection Bar can show "3 skids pending change, reconciliation does not balance"
After save	Reconciled rows clear their pending marker; unresolved warnings are logged

Asynchronous Contracts

Threading rules

Rule	Reason
All view-model async methods resume on the UI thread (<code>ConfigureAwait(true)</code> , the default)	Property notifications and <code>ObservableCollection</code> mutations must occur on the UI thread
Never use <code>.Result</code> or <code>.Wait()</code> on any validation or persistence call	Both deadlock against the UI <code>SynchronizationContext</code>
No validator performs synchronous database work	A frozen receiving station during a 40-row batch is a production stoppage
Use <code>DispatcherQueue.TryEnqueue</code> only from genuine background continuations	The lookup control's own implementation is the reference pattern
Bound the work: one in-flight request per field, one in-flight batch operation total	Prevents request pile-up when the operator types quickly

Cancellation surfaces

Operation	Cancellable	Behavior on cancel
Field validation	Yes	Discard result, clear inline status, leave Apply state unchanged
Batch apply to in-memory rows	No	Sub-millisecond operation on held objects; cancellation adds no value
Save / persist	No	<code>SaveAsync</code> semantics are unchanged; the operator sees the existing busy state
Reprint queueing of N rows	No, but progress-reported	Report <code>Queueing 12 of 40</code> ; a failed row never aborts the remaining rows
LabelView launch	No	Single shell invocation

Busy state

`ViewModel_Shared_Base.IsBusy` is the single busy flag. Existing controls depend on it (`ProgressRing` on the Reprint page, footer state in Edit Mode). Batch work must set it and must always clear it in a `finally` block, exactly as `SaveAsync` and the load commands already do. Long queueing loops must report progress rather than leaving a static spinner for thirty seconds.

Partial Failure and Blocked Work

The rule

A batch operation never partially applies silently, and a partial failure never loses the operator's work.

Persistence failures

`IService_MySQL_Receiving.UpdateReceivingLoadsAsync` and `UpdateCurrentLabelDataAsync` return counts, not per-row results. Two safeguards are therefore required at the UI layer:

1. **Pre-flight revalidation.** Immediately before the write, re-verify the batch against the same rules the overlay used, so a row that became invalid while the dialog was open is caught.
2. **Count reconciliation.** Compare the returned updated count against the requested count. A mismatch is reported to the operator as a partial write, with the difference quantified, not silently accepted. The existing `SaveAsync` path already computes `requestedLabelUpdates` versus

`labelUpdated` and takes the max; that heuristic smooths over a real discrepancy and should be replaced with an explicit mismatch report once batch sizes grow.

Reprint partial failures

`Model_ReprintBatchResult` already buckets `Queued`, `AlreadyQueued`, and `Failed`. Surface all three as counts plus a details list, and offer `Retry failed (N)` so the operator repairs the run without re-selecting anything. Failed rows keep their checked state.

Preserving work

Situation	Guarantee
Validation blocks apply	Selection and typed values remain in the dialog
Apply succeeds, save fails	In-memory rows still hold the change; the operator can retry save
Save partially succeeds	The operator is told which rows did not persist
App closes with pending changes	No database write occurred; this is existing behavior and must stay true
Reprint partially fails	Failed rows remain selected and the batch handoff remains active

Error Escalation

`IService_ErrorHandler` is the established escalation path. Batch operations must reuse it rather than inventing dialogs.

Situation	Call	Severity
Field validation finds a blocker	Inline pane only	Not escalated
Apply blocked by ownership rules	<code>ShowErrorDialogAsync</code>	Warning
Batch apply succeeded	<code>ShowErrorDialogAsync</code> summary	Info
Partial write persisted	<code>ShowErrorDialogAsync</code> with the count difference	Warning
Validation service unreachable	<code>HandleErrorAsync</code> with an explicit "could not verify" message and a retry path	Error
Save threw	<code>HandleErrorAsync</code>	<code>Critical</code> , with the exception, matching <code>SaveAsync</code>
Label launch failed	<code>ShowErrorDialogAsync</code> with the message from <code>Service_LabelViewLauncher</code>	Warning

A validator that cannot reach its data source must never default to "valid". Unverifiable is a distinct outcome from verified, and it surfaces as `Error` with a retry path.

Message Catalog

Exact operator-facing copy. Keep it plain, specific, and free of error codes.

Key	Text
<code>Batch.SelectionHeadline</code>	"3 skids selected"
<code>Batch.ReconciliationBalanced</code>	"Balances to 154,540 lb"

Key	Text
Batch.ReconciliationShort	"Short by 18,040 lb – selected 136,500 lb, expected 154,540 lb"
Batch.ReconciliationOver	"Over by 82,094 lb – selected 236,634 lb, expected 154,540 lb"
Batch.ReconciliationOutOfTolerance	"This does not balance. Confirm you want to apply it anyway."
Batch.BlockerPartNotFound	"MMC00076 is not an exact part number. Check the value or cancel."
Batch.ConfirmFuzzyPart	"The part was matched approximately. Confirm MMC000767 is correct."
Batch.WarnHeatLotBlank	"Heat / Lot is blank. It will be saved as \"Nothing Entered\"."
Batch.InfoNoChange	"No change for 2 of 3 skids."
Batch.BlockNegativeQuantity	"This would make 1 skid negative (V-F0-03: -12,000 lb)."
Batch.BlockOwnership	"1 selected skid is owned by another user and cannot be changed here."
Batch.PartialWrite	"14 of 15 skids were saved. 1 skid did not update. It is still marked as changed."
Batch.Unverifiable	"Could not verify the part number. Nothing was applied. Try again."
Reprint.QueuedOnly	"3 labels queued. Label opened in LabelView."
Reprint.AlreadyQueuedWithOwner	"Queued on 10/1/2026 4:12 PM by mlaurin."
Reprint.PartialQueue	"2 of 3 labels queued. 1 failed. Retry failed rows?"

These belong under the settings keys pattern used across the app, not inline in XAML, so wording can be corrected without a code change.

Logging

Event	Level	Content
Batch opened	Info	Row count, source mode, source part and target part
Validation finding	Info or Warning by severity	Field, severity, row count, not the operator's typed value if it is sensitive
Batch applied to memory	Info	Row count, changed field count
Batch persisted	Info	Requested versus returned update count
Partial write	Warning	The exact difference and affected identifiers
Reconciliation out of tolerance	Warning	Selected total, expected total, variance
Reprint queue result	Info	Queued, already queued, failed counts plus batch id

Log the batch id on every line in the sequence so a support call can reconstruct one operator's run from the log alone. That is the entire point of the `BatchId` on the handoff model.

Test Expectations

Following the repository testing rules (FluentAssertions, `MethodName_ShouldResult_WhenCondition` , `IAsyncLifetime` for integration, `TEST-` data prefix).

Test	Type	Asserts
ApplyBatch_ShouldUpdateEverySelectedRow_WhenOverrideIsTicked	Unit	Field application semantics
ApplyBatch_ShouldLeaveUntickedFieldUnchanged_WhenValueDiffersAcrossRows	Unit	Override contract

Test	Type	Asserts
Reconciliation_ShouldReportShort_WhenSelectedTotalIsBelowExpected	Unit	Variance math and severity
Reconciliation_ShouldAllowApply_WhenNoExpectedTotalEntered	Unit	Neutral default
ValidateBatch_ShouldBlock_WhenPartLookupReturnsNoExactMatch	Unit	Block severity gate
ValidateBatch_ShouldRequireAcknowledgment_WhenLookupUsesFuzzyFallback	Unit	The fuzzy guard
ValidateBatch_ShouldCallRestrictedPartOnce_WhenManyRowsShareTheNewPart	Unit	No N+1 validation calls
ValidateBatch_ShouldDiscardStaleResult_WhenFieldChangesMidFlight	Unit	Cancellation and stale guard
SaveAsync_ShouldReportPartialWrite_WhenReturnedCountIsLessThanRequested	Unit	Count reconciliation
BatchEdit_ShouldPersistOnlyOnSave_WhenDialogAppliesChanges	Integration	No write before save
BatchEdit_ShouldSurvivePagination_WhenSelectionSpanningPagesIsUsed	Integration	Selection durability
Reprint_ShouldOfferRetryForFailedOnly_WhenPartialQueueFailureOccurs	Unit	Partial-failure recovery

Acceptance Criteria

- ☐ Every field in the batch overlay shows inline status within 300ms of typing stopping.
- ☐ No Block finding allows Apply; every Confirm finding requires one acknowledgement per batch.
- ☐ A fuzzy part match never applies without acknowledgement.
- ☐ Blockers list every affected row identifier, not just a count.
- ☐ The UI thread is never blocked: no .Result , no .Wait() , no synchronous data access in any validator.
- ☐ A validation service outage surfaces as Error with a retry path and applies nothing.
- ☐ Partial persistence writes are reported with the exact count difference and leave unpersisted rows marked as changed.
- ☐ Partial reprint failures offer Retry failed (N) and preserve failed rows' checked state.
- ☐ All operator-facing strings are settings-driven.